

IN5020 Assignment 2 Report

Group 8 IN5020 - Distributed Systems

Members:

borgebj | Børge

vegarhje | Vegard

sellebaf | Selleban

User Guide

A detailed ReadMe is contained within the zipped folder that was delivered. Within it, all the necessary information needed to properly compile and run the Replicated bank account application solution is provided. A simple guide for how to run the spread toolkit is also provided.

Workload

We first started off the assignment by going setting up the project, and establishing proper client server connection as a group. From there we worked together on sending a simple spread message to the replicas and scheduling the broadcasts to send every 10 seconds. At this point members of the group were working more independently taking on different tasks of building out the technical features e.g. defining commands, building transaction instances to send, processing received messages, or parsing. Once a lot of the solution was in place we went back to doing a lot of the development and testing together again, discussing implementations such as the `getSyncedBalance` function and finishing off the smaller details.

GetSyncedBalance implementation

We made sure our solutions for the GetSyncedBalance implementations were very faithful to the assignment description, as there probably a different ways to implement said function, especially the non naive implementation.

Naive Implementation

The naive implementation is called loops as long as there are still transactions in the outstanding list. When the outstanding list is empty the balance is returned and the printed balance is displayed to the client that called it.

Non Naive implementation

The non naive implementation adds GetSyncedBalance to the outstanding list to be broadcast. From there the only the listener who corresponds with the sender ID executes the function, returns the balance. Finally the transaction instance is removed from the outstanding list and NOT added to the execution list per the requirements.

Testing and observations

The first difference we see is in the timings of the GetSyncedBalance printoutput. The Naive getSyncedBalance prints when the outstanding list is empty, this works independently of the broadcast of transactions. This means the balance will be printed after every transaction within outstanding collection has been executed. But it also creates another problem in that if GetSyncedBalance is called and followed by a set of transactions it will give the updated balance after those transactions as opposed to the balance when it was called with every previous transaction to its call executed. This is the main difference in the non naive version it accounts for every previous action prior to its called which can cause the balance in certain scenarios to output different from the naive version.